Required Presentation Components (6–10 slides):

Smart Home Controll Center

Team 6

- Michael Hawes
- Evan McQueary
- Jacques Ndayisaba
- Josh Spencer

Problem Statement

Issues with Commercial Smart Home Systems

1. **Cloud Dependency**: Data sent to third-party servers, privacy concerns
2. **Proprietary Ecosystems**: Different devices = different apps
3. **Vendor Lock-in**: High costs, forced upgrades
4. **Poor Interoperability**: Devices from different brands can't talk

Our Solution: Open-source, local-only control with MQTT protocol and unified interface

System design summary

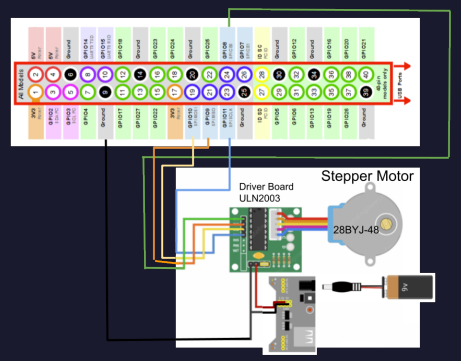
Control Center

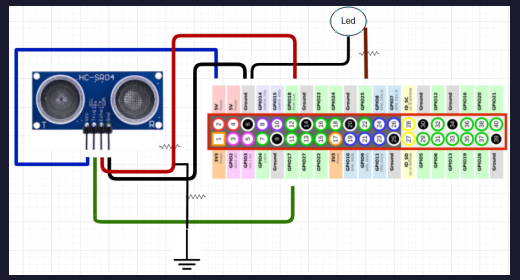
- MQTT broker communication for device messaging
- Real-time temperature and humidity monitoring via SHT41 sensor
- User interface for device control and monitoring

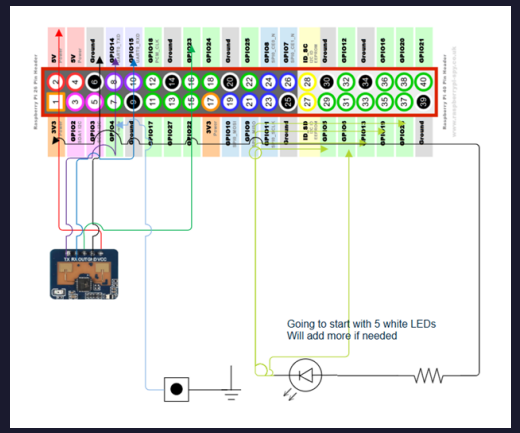
End Devices

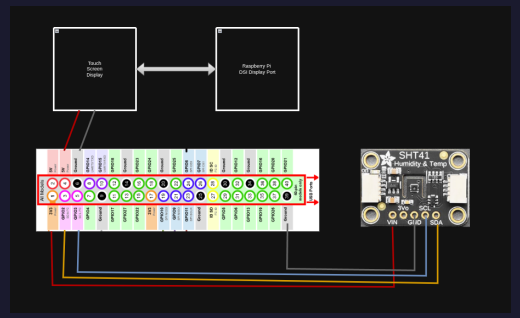
- Alarm System using Presence detection via LD2410C radar sensor
- Smart Vent using Servo Motors
- Smart Window shades
- Lights with motion detection

Wiring diagrams

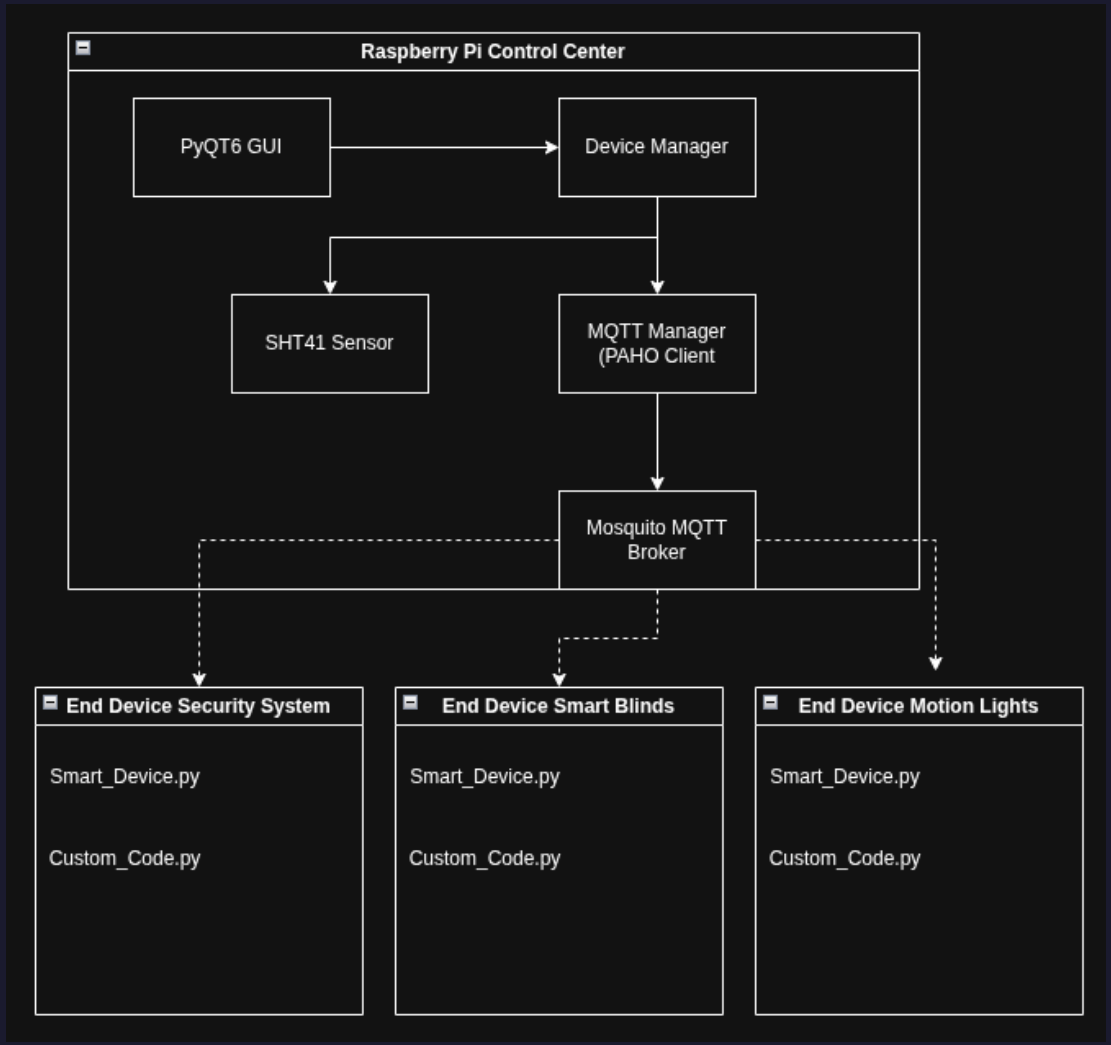








Architecture diagram



Code highlights

Control Center Architecture

- **MqttManager**: Paho MQTT client bridging broker to PyQt6 signals
- **DeviceManager**: Device registry + state tracking + sensor integration and Thread Control
- **DevicePanel**: Scrollable, touch-friendly device control UI
- **EnvironmentPanel**: Real-time temp/humidity display

Control Center Features

- **Full-screen touch interface** for Raspberry Pi display
- **Auto-discovery**: Devices announce themselves via MQTT
- **One-tap control**: Toggle devices on/off instantly
- **Live sensor readout**: Temperature & humidity updates every 15 seconds
- **Status indicators**: Online/offline and on/off state for each device
- **Responsive layout**: Scales to any screen resolution

SmartDevice Base Class

- **Reusable framework** for all end devices (lights, vents, locks, etc.)
- Handles MQTT connection, auto-announcement, LWT (Last Will Testament)
- Manages command subscriptions and state publishing
- **Subclasses only implement:**
  - `handle_command()`: React to control panel commands
  - `on_ready()`: Initialize hardware (GPIO, sensors)

Adding a New Device

Inherit from `SmartDevice`, override 2 methods → device is ready to go

- Control panel auto-discovers it via MQTT announcement
- Configure the control logic of the end device (Only On/Off functionality currently)

Software & Hardware

- **Mosquitto**: MQTT broker (lightweight pub/sub messaging)
- **Python 3**: Backend + sensor drivers
- **PyQt6**: Cross-platform GUI framework
- **Paho-MQTT**: Python MQTT client library
- **Raspberry Pi**: Control center host
- **SHT41**: High-accuracy temp/humidity sensor (I2C)
- **LD2410C**: Human presence radar sensor
- **Servo Motors**: Smart vent actuation

Live Demo With Q&A

Future Improvements

- **Advanced control logic**: Dimming, temperature setpoints, automation rules
- **More end devices**: Integration with smart locks, thermostats
- **Zigbee protocol**: Low-power mesh networking for battery devices
- **Home Assistant Integration:** Integrate with the Home assistant ecosystem
- **Cloud backup**: Logs and automation scheduling
- **3D printed casings**: Professional enclosures for all devices